Matplotlib and Seaborn

Data Visualization:
Core Plotting Functions and Workflow

DAEN 300

Fall 2026



Goals

- Build plots step by step rather than copying finished examples.
- Recognize the main Matplotlib plotting functions and when to use them.
- Use the object-oriented interface for clearer, reusable code.
- Add titles, labels, legends, ticks, colorbars, and annotations.
- Use Seaborn for higher-level statistical plots.

Notebook: Concept - Your Turn - Check

Pattern

Each section contains a short concept note, a stubbed code cell, and a quick check cell.

```
# Your turn
x = np.linspace(0, 2*np.pi, 100)
# TODO: plot sin(x)

# Quick check
if len(plt.gca().lines) >= 1:
    print("OK: line created")
else:
    print("Try again: no line found")
```

Essential setup

- `numpy` creates numeric arrays.
- `pyplot` is the common Matplotlib interface.
- Run setup after restarting the kernel.

```
import numpy as np
import matplotlib.pyplot as plt

# Optional style
plt.style.use("seaborn-v0_8-whitegrid")
```

Two plotting interfaces

Stateful pyplot

```
plt.figure()
plt.plot(x, y)
plt.xlabel("x")
plt.ylabel("y")
plt.show()
```

Object-oriented

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set(xlabel="x", ylabel="y
    ")
plt.show()
```

Prefer the object-oriented for multi-step figures.

Line plots: trends over ordered values

- Main function: `plot()`.
- Useful arguments: `color`, `linestyle`, `linewidth`, `label`.
- Add context with titles, labels, and limits.

```
fig, ax = plt.subplots()
ax.plot(x, np.sin(x), label="sin(x)")
ax.set(title="Sine wave", xlabel="x", ylabel="sin(x)")
ax.legend()
```

Scatter plots: relationships between variables

- Main functions: `plot(..., marker=...)` and `scatter()`.
- `scatter()` supports per-point encodings: size, color, alpha.
- Use scatter plots when observations are independent points.

```
fig, ax = plt.subplots()
ax.scatter(x, y, s=sizes, c=colors, alpha=0.7)
ax.set(xlabel="x", ylabel="y", title="Encoded scatter")
```

Uncertainty: error bars and bands

- `errorbar()` shows point estimates with uncertainty intervals.
- `fill_between()` creates continuous uncertainty bands.
- Use alpha/transparency so the main trend remains visible.

```
fig, ax = plt.subplots()
ax.errorbar(x, y, yerr=dy, fmt="o")

ax.plot(x_grid, y_fit)
ax.fill_between(x_grid, y_fit-dy, y_fit+dy, alpha=0.2)
```

Density, contour, and image displays

- `contour()` draws level curves.
- `contourf()` fills regions between levels.
- `imshow()` displays a 2D array as an image.
- Pair color-encoded plots with `colorbar()`.

```
fig, ax = plt.subplots()
contour = ax.contourf(X, Y, Z, levels=20)
fig.colorbar(contour, ax=ax)
```

Histograms and binned density

- `hist()` summarizes one numeric variable.
- `hist2d()` bins paired observations on a grid.
- `hexbin()` is useful for dense scatter data.

```
fig, ax = plt.subplots()
ax.hist(data, bins=30)

fig, ax = plt.subplots()
h = ax.hexbin(x, y, gridsize=30)
fig.colorbar(h, ax=ax)
```

Legends: mapping symbols to meaning

- Add `label=` when plotting.
- Call `legend()` after all labeled artists are created.
- For selected entries, pass handles and labels manually.

```
fig, ax = plt.subplots()
line1, = ax.plot(x, np.sin(x), label="sine")
line2, = ax.plot(x, np.cos(x), label="cosine")
ax.legend()

# selected entries only
ax.legend([line1], ["sine only"])
```

Colorbars and colormaps

- Use `cmap=` to choose the mapping from values to colors.
- Use `vmin`/`vmax` or `clim()` for comparable scales.
- Use sequential maps for magnitude; diverging maps for deviation around a midpoint.

```
fig, ax = plt.subplots()
im = ax.imshow(I, cmap="RdBu", vmin=-1, vmax=1)
fig.colorbar(im, ax=ax)
```

Multiple subplots

- `subplot()` selects one axes in a grid.
- `subplots()` creates a figure and axes array together.
- `GridSpec` supports custom layouts and spanning cells.

```
fig, axes = plt.subplots(2, 3, figsize=(8, 5))
axes[0, 0].plot(x, np.sin(x))
axes[0, 1].plot(x, np.cos(x))

fig.tight_layout()
```

Text and annotation

- `text()` places free text on an axes.
- `annotate()` connects text to a data point, often with an arrow.
- Use annotations sparingly; highlight the one point that matters.

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.annotate("maximum", xy=(x0, y0), xytext=(x1, y1),
            arrowprops={"arrowstyle": "->"})
```

Ticks and formatting

- Locators control tick positions.
- Formatters control tick labels.
- Hide ticks or labels when they distract from the message.

```
from matplotlib.ticker import MultipleLocator,
    FuncFormatter

ax.xaxis.set_major_locator(MultipleLocator(np.pi / 2))
ax.xaxis.set_minor_locator(MultipleLocator(np.pi / 4))
ax.xaxis.set_major_formatter(FuncFormatter(pi_formatter))
```

Styles and runtime defaults

- `plt.style.use()` applies a style globally.
- `plt.style.context()` applies a style temporarily.
- `plt.rc()` and `rcParams` adjust defaults.

```
plt.rc("lines", linewidth=2)

with plt.style.context("ggplot"):
    fig, ax = plt.subplots()
    ax.plot(x, y)
```

Three-dimensional plotting

- Create 3D axes with `projection="3d"`.
- Common methods: `plot3D()`, `scatter3D()`, `contour3D()`.
- Surfaces: `plot_wireframe()`, `plot_surface()`, `plot_trisurf()`.

```
fig = plt.figure()
ax = fig.add_subplot(111, projection="3d")
ax.plot3D(x, y, z)
```

Seaborn: statistical plotting layer

- Built on Matplotlib; works naturally with pandas DataFrames.
- Main functions in this session: `kdeplot()`, `pairplot()`, `FacetGrid`, `catplot()`, `jointplot()`.
- Use Seaborn when the statistical summary is the focus.

```
import pandas as pd
import seaborn as sns

sns.kdeplot(data=df, x="value")
sns.pairplot(df, hue="group")
sns.catplot(data=df, x="category", y="value", kind="box")
```

Function map

Need	Main functions
Line trends	`plot`, `set`, `legend`
Point relationships	`scatter`, `plot(..., marker=)`
Uncertainty	`errorbar`, `fill_between`
2D fields	`contour`, `contourf`, `imshow`, `colorbar`
Distributions	`hist`, `hist2d`, `hexbin`, `kdeplot`
Layouts	`subplot`, `subplots`, `GridSpec`
Explanatory text	`text`, `annotate`
Ticks/styles	`MultipleLocator`, `FuncFormatter`, `style`, `rc`
3D	`projection="3d"`, `plot3D`, `plot_surface`
Seaborn grids	`pairplot`, `FacetGrid`, `jointplot`, `catplot`

Checks without tracebacks

- `assert` is useful, but traceback output can distract beginners.
- Print what was found so you can debug notebook state in the future.

```
line_count = len(plt.gca().lines)

if line_count >= 1:
    print(f"OK: found {line_count} line(s).")
else:
    print("Try again: no plotted line found yet.")
```

Hints

- Separate data prep from plotting commands.
- Build figures in layers: data, labels, legends, scales, annotations.
- Use explicit `fig, ax` references for reliable notebook behavior.
- Read function signatures and documentation.
- Always ask youself:
  what comparison should this figure help the viewer make?!

Assignment

1. The goal is to finish this assignment during class today.
2. If you need more time, I can grant an extension, but whatever you've completed by the end of class must be submitted for credit.
3. Any work you do after that is optional and may earn extra points, but I'm not guaranteeing it.
4. After you submit the assignment, take the 11-question quiz on Canvas to check your understanding.